MUBD

Màster Universitari en Enginyeria de Dades Massives (Big Data)

Estadística

Índice

Métodos de clustering

01 Introducción

02 Clusterización jerárquica

- Distancia, inercia y criterio de Ward
- Algoritmo aglomerativo y partición

03 K-means

- Algoritmo, parámetros y convergencia
- Métodos y variantes

04 Medidas de rendimiento

05 Sistemas mixtos

CLUSTERING

Introducción

01

Introducción

¿Qué busca el clustering?

- El clustering agrupa observaciones según la similitud de sus características.
- El objetivo es que las observaciones de un mismo grupo **se parezcan entre sí y se diferencien de las de otros grupos.**
- Es aprendizaje **no supervisado**: los datos no incluyen etiquetas que indiquen a qué grupo debe pertenecer cada observación.
- Ejemplo: agrupar clientes con hábitos de compra parecidos para identificar segmentos de consumidores.

Clustering y clasificación: qué los diferencia

Grupos descubiertos frente a clases conocidas

- **Clustering: agrupar por similitud (no supervisado)**
  - Descubre grupos de observaciones con características similares.
  - Aprende sin usar etiquetas de clase previamente asignadas.
  - Ejemplo: agrupar flores por sus medidas, sin conocer su especie.

- **Clasificación: predecir una clase (supervisado)**
  - Aprende de ejemplos con una etiqueta o clase conocida.
  - Predice a qué clase pertenece una nueva observación.
  - Ejemplo: predecir la especie de una flor nueva con sus medidas.
  - Algoritmos: Random Forest, KNN y SVM (próximas sesiones).

Clustering vs. clasificación

Una misma temática, dos objetivos

Clustering

Descubrir grupos sin etiquetas

¿Qué perfiles de clientes aparecen?
Agrupa por gasto y frecuencia.

¿Qué flores se parecen?
Agrupa por medidas, sin la especie.

¿Qué temas aparecen?
Agrupa noticias sin categorías previas.

¿Qué máquinas se parecen?
Agrupa por sus señales de vibración.

¿Qué canciones se parecen?
Agrupa por sonido, sin usar géneros.

Clasificación

Aprender con etiquetas y predecir clases

¿Comprará un cliente nuevo?
Aprende de casos «sí / no».

¿De qué especie es esta flor?
Aprende de especies conocidas.

¿Es deporte o economía?
Aprende de noticias etiquetadas.

¿Normal o avería?
Aprende de estados conocidos.

¿Es jazz o rock?
Aprende de géneros conocidos.

CLUSTERING

Clusterización jerárquica

02

Clusterización jerárquica

Introducción

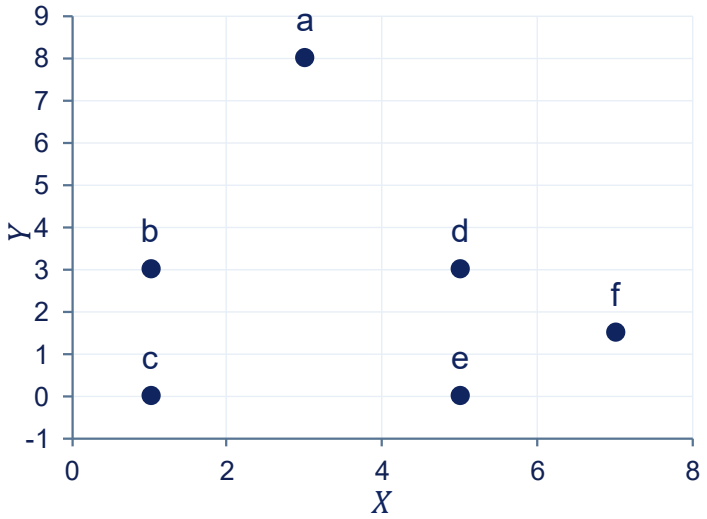
- **Ventajas**
  - No es imprescindible tener variables cuantitativas (aunque es más usual) ya que se pueden definir distancias para variables categóricas.
  - No requiere definir el número de agrupaciones a priori. Se calcula un árbol jerárquico independientemente de los grupos.
  - El árbol permite visualizar de forma intuitiva distancias entre individuos para tamaños muestrales no muy grandes.

- **Inconvenientes**
  - Requiere definir un tipo de distancia.
  - Requiere el cálculo de todos los pares de distancias. Mayor tiempo de computación para muestras grandes (>500).

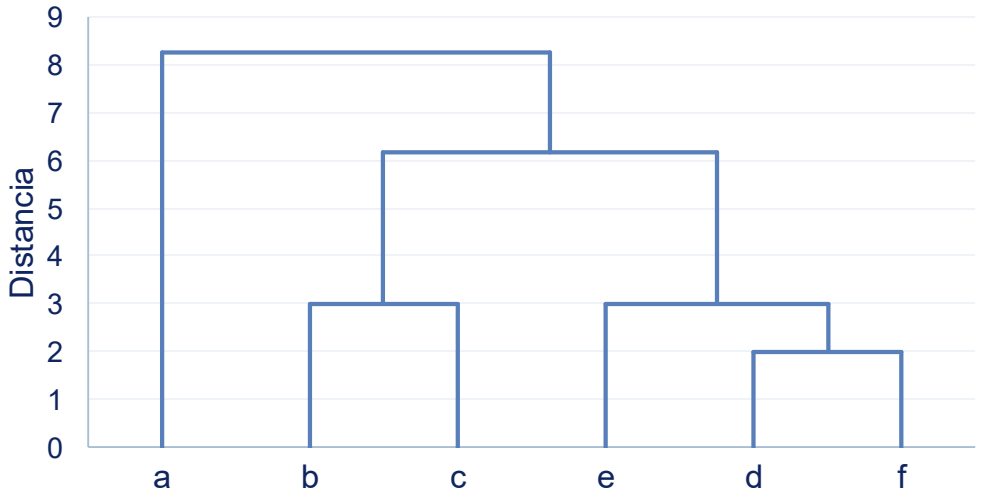
Clusterización jerárquica

Distancias entre observaciones y grupos

Coordenadas X e Y



Fusiones según la distancia completa



Distancias euclídeas
entre observaciones

	A	B	C	D	E	F
A						
B	5,39					
C	8,25	3				
D	5,39	4	5			
E	8,25	5	4	3		
F	7,63	6,18	6,18	2	2,5	

Distancias entre grupos
A, BC, DF y E

	A	BC	DF	E
A				
BC	8,25			
DF	7,63	6,18		
E	8,25	5	3	

Distancias entre grupos
A, BC y DEF

	A	BC	DEF
A			
BC	8,25		
DEF	8,25	6,18	

Distancias entre grupos
A y BCDEF

	A	BCDEF
A		
BCDEF	8,25	

Clusterización jerárquica

Dendrograma: similitud entre futbolistas

Qué define la similitud

Selección nacional: 50 %

Posición: 20 %

Liga de referencia: 10 %

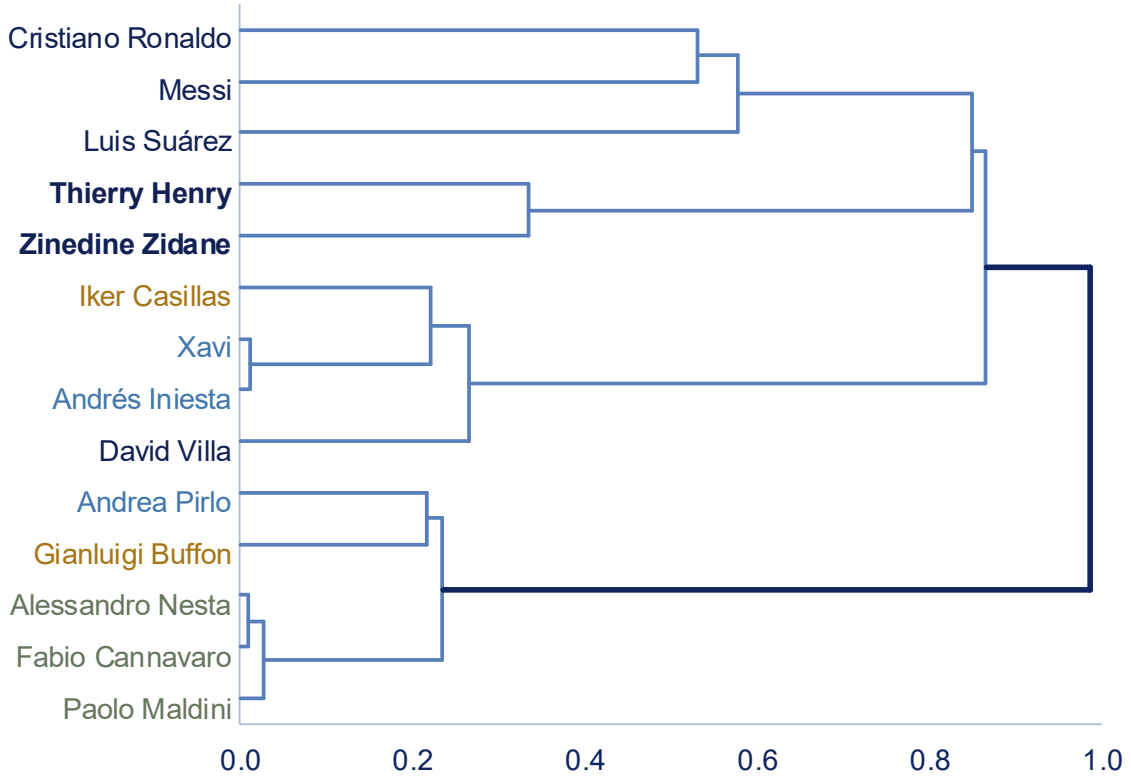
Goles con selección: 15 %

Edad: 5 %

Henry y Zidane

Comparten selección: Francia. Con estos pesos se unen antes que a otros jugadores.

El árbol termina en un único clúster con los 14 jugadores.



El dendrograma es la representación gráfica más habitual de la clusterización jerárquica. En este gráfico, el eje horizontal muestra la distancia de fusión: menor distancia significa mayor similitud.

Clusterización jerárquica

Proceso

Al realizar una clusterización jerarárquica, se deben realizar los siguientes pasos (en azul, decisiones a tomar):

1. **Decidir el tipo de distancia a emplear (euclídea, manhattan…)**
2. Construir la matriz de disimilitudes
3. **Decidir el método de agrupación (aglomerativo o divisivo)**
4. **Decidir el sub-método de agrupación (Ward, completo…)**
5. Aplicar el algoritmo de clusterización
6. **Decidir el criterio para realizar la partición de los clústeres**
7. Particionar la muestra

NOTA: Con los pasos 1 a 5 construimos el dendograma.

Distancias entre observaciones

Tipos (Paso 1)

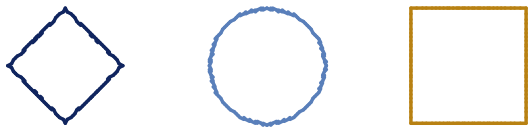
Minkowski $d(i,l) = \left(\sum_{k=1}^{K} |x_{ik} - x_{lk}|^p\right)^{\frac{1}{p}}$

Manhattan ($p = 1$) $d(i,l) = \sum_{k=1}^{K} |x_{ik} - x_{lk}|$

Euclídea ($p = 2$) $d^2(i,l) = \sum_{k=1}^{K} (x_{ik} - x_{lk})^2$

Máxima ($p = \infty$) $d(i,l) = \max_k \|x_{ik} - x_{lk}\|$

Contornos de distancia unidad



$p = 1$ $p = 2$ $p = \infty$

Mahalanobis $d^2(i,l) = (x_i - x_l)^{\mathrm{T}} \Sigma^{-1} (x_i - x_l)$

Σ: matriz de varianzas y covarianzas

Canberra $d(i,l) = \sum_{k=1}^{K} \frac{|x_{ik} - x_{lk}|}{|x_{ik} + x_{lk}|}$

Gower

Variables categóricas

$d_k(i,l) = \sum_{k=1}^{n} d_k(i,l) \rightarrow d_k(i,l) = \begin{cases} 1 \; si \; i_k \neq j_k \\ 0 \; si \; i_k = j_k \end{cases}$

Índice de Jaccard $d(i,l) = \frac{\boldsymbol{n_{++}}}{\boldsymbol{n_{++}} + \boldsymbol{n_{+-}} + \boldsymbol{n_{-+}}}$

Variables categóricas

Distancias entre grupos de observaciones

Tipos (Paso 4)

- ***Simple.*** Mínima distancia entre dos puntos pertenecientes uno a cada grupo.
- ***Completa.*** Máxima distancia entre dos puntos pertenecientes uno a cada grupo.
- ***Entre centros de gravedad.*** Distancia entre los centros de gravedad.
- ***Criterio de Ward.*** No es una distancia en sí, sino un criterio para hacer la agrupación que consiste en minimizar la **inercia INTRA-GRUPO**.

Inercia

Dispersión total, entre grupos y dentro de cada grupo

- La inercia mide la dispersión: suma las distancias al cuadrado respecto a un centro.
- El centroide es la media de las observaciones. Consideramos Q grupos, con I_q observaciones en cada grupo y K variables:

Inercia Total	$\sum_{k=1}^{K}\sum_{q=1}^{Q}\sum_{i=1}^{I_q}(y_{iqk}-\bar{y}_k)^2$	*Dispersión respecto al centro global.*
Inercia Entre-grupo	$\sum_{k=1}^{K}\sum_{q=1}^{Q} I_q(\bar{y}_{qk}-\bar{y}_k)^2$	*Separación de centroides, ponderada por tamaño.*
Inercia Intra-grupo	$\sum_{k=1}^{K}\sum_{q=1}^{Q}\sum_{i=1}^{I_q}(y_{iqk}-\bar{y}_{qk})^2$	*Dispersión dentro de cada grupo.*

- Menor inercia intra-grupo significa grupos más compactos.

$$\text{Inercia Total} = \text{Inercia Entre}-\text{grupo} + \text{Inercia Intra}-\text{grupo}$$

Criterio de Ward

Agrupación

- Se parte de Q clústeres
- Se desea una agrupación ideal que pase a $Q - 1$ clústeres
- Sea el clúster p (con centro de gravedad g_p y tamaño I_p) y el clúster q (con centro de gravedad g_q y tamaño I_q). El incremento en la inercia intra-grupo se cuantifica por:

$$\Delta(p, q) = \frac{I_p I_q}{I_p + I_q} d^2(g_p, g_q)$$

- El criterio de Ward consiste en juntar aquellos clústeres tales que minimicen el incremento en la inercia intra-grupo al pasar de Q a $(Q - 1)$ clústeres.
- Este criterio favorece juntar:
  - Los clústeres cercanos
  - Los clústeres pequeños

Método de agrupación

AGNES y DIANA: dos formas de construir la jerarquía

AGNES: aglomerativo

AGglomerative NESting · Ascendente

Inicio: cada observación forma su propio clúster. El proceso une grupos.

1. Construye la matriz de distancias entre las observaciones.

2. Une los dos grupos más próximos según el enlace (p. ej., Ward).

3. Registra la altura de fusión y actualiza las distancias del nuevo grupo al resto.

4. Repite hasta reunir todas las observaciones en un único grupo (raíz).

DIANA: divisivo

DIvisive ANAlysis · Descendente

Inicio: todas las observaciones forman un solo clúster. El proceso divide grupos.

1. Selecciona el grupo más heterogéneo, con mayores diferencias internas.

2. Lo divide en dos subgrupos y separa los objetos menos similares.

3. Repite sobre los grupos resultantes para obtener agrupaciones más pequeñas.

4. Termina con un clúster por observación, completando la jerarquía.

La agrupación aglomerativa es buena para identificar pequeños grupos y la divisiva sirve para identificar grandes agrupaciones.

Clusterización jerárquica

Algoritmo aglomerativo

1. **Construir la matriz de distancias**
   Calcula las distancias entre todas las observaciones.

2. **Elegir y unir los dos grupos**
   Aplica el criterio de enlace para elegir la pareja, por ejemplo Ward.

3. **Representar la fusión en el dendrograma**
   La altura corresponde a la distancia o al cambio de inercia, según el criterio.

4. **Actualizar la matriz de distancias**
   Sustituye las filas y columnas de los grupos unidos por las del nuevo grupo y recalcula sus distancias a los demás.

5. **Repetir el proceso**
   Vuelve al paso 2 hasta reunir todas las observaciones en un único grupo (raíz).

Calidad del árbol jerárquico

Distancia y correlación cofenética

- Para evaluar la calidad del árbol (dendograma) construido se deben comparar cuán similares son las distancias (alturas) producidas por el dendograma respecto a las distancias originales.
- Las distancias obtenidas del dendograma se denominan **distancias cofenéticas**.
- Para evaluar la calidad, se calcula la **correlación cofenética** entre estas distancias y las distancias originales.
- Si la agrupación es válida, debe existir una fuerte correlación entre estas distancias. Se considera que los valores superiores a 0.75 son aceptables.
- Escoger distancias “medias” para la agrupación produce valores altos de esta correlación.

Clusterización jerárquica

Número de clústeres

- Una vez se tiene el dendograma, se debe definir una partición en K clústeres.
- Criterios para escoger una partición:
  - **Visual**. A partir del dendograma
  - **Cambio de inercia**
    - Regla del codo.
    - Minimizar el cociente $\frac{\Delta(q)}{\Delta(q+1)}$ donde $\Delta(q)$ es el cambio en la inercia intra al pasar de q a $q-1$ clústeres
  - **Parsimonia**. No es conveniente escoger un gran número de clústeres
  - **Interpretabilidad**. Los grupos deben tener algún sentido

Tendencia a la agrupación

Estadístico de Hopkins

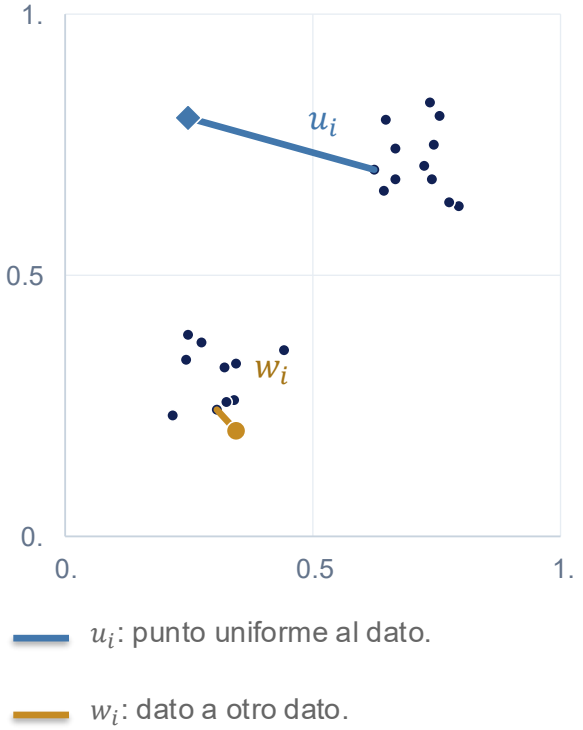
1. Escoger n puntos aleatorios de los datos D: $D' = (p_1, \dots, p_n) \subset D$.

2. Para cada $p_i \in D'$, buscar su vecino más próximo $p_j \in D$, excluyendo el propio punto, y calcular la distancia w_i.

3. Generar $Q = (q_1, \dots, q_n)$ con n puntos uniformes y el mismo rango de valores que los datos D.

4. Para cada $q_i \in Q$, buscar su vecino más próximo en los datos reales D y calcular la distancia u_i.

Vecinos más próximos



Cálculo del estadístico de Hopkins

$$H = \frac{\sum u_i}{\sum w_i + \sum u_i}$$

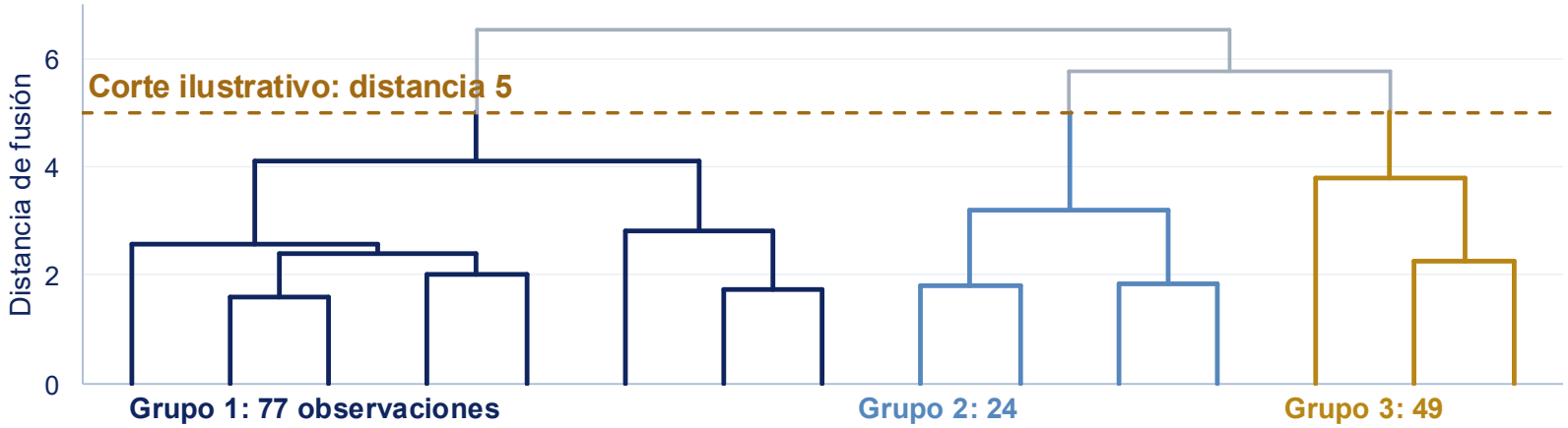
$H \approx 0.5$: compatible con azar. $H \approx 1$: tendencia a formar grupos.

Clusterización jerárquica

Python: construir el árbol y obtener los grupos

Implementación

```
from scipy.cluster.hierarchy import (
     linkage, dendrogram, fcluster)
import matplotlib.pyplot as plt
import numpy as np
Z = linkage(X, method="complete")  # Enlace completo.
dendrogram(Z, truncate_mode="lastp", p=15)  # 15 hojas resumen el árbol.
  # Una hoja puede reunir varios casos.
plt.show()  # Cada color identifica un grupo.
lab_h = fcluster(Z, 3, criterion="maxclust")  # Las etiquetas son arbitrarias.
print(np.bincount(lab_h)[1:])
[77 24 49]
```



El corte cruza 3 ramas: define 3 grupos. Una fusión más baja indica mayor similitud.

CLUSTERING

K-means

03

K-means

Introducción

Tipos de variables

Preferentemente cuantitativas u ordinales con muchas categorías. Para otros tipos, se requieren adaptaciones de la distancia o del método.

Ventajas No calcula todas las distancias por pares. Por tanto, requiere menos tiempo de cómputo que la clusterización jerárquica.

Inconvenientes

1. **Clústeres no convexos**
   Puede producir agrupaciones inadecuadas.
2. **Sensibilidad al inicio**
   Depende de los centros iniciales, sobre todo con pocos datos.
3. **Número de grupos prefijado**
   Hay que escoger k antes del ajuste.
4. **Sensibilidad a los outliers**
   Al basarse en medias, los valores extremos pueden alterar el resultado.

Posibles soluciones

- Usar variantes de K-means que reduzcan este problema.
- Probar distintos puntos iniciales y comparar los resultados.
- Probar varios valores de k si el coste computacional lo permite.
- Probar métodos más robustos: K-medians o K-medoids.

K-means

Algoritmo paso a paso

Objetivo: minimizar la inercia intragrupo
Distancias al cuadrado al centroide.

$$J = \sum_{j=1}^{k} \sum_{x_i \in C_j} \left\| x_i - \mu_j \right\|^2$$

x_i: observación C_j: grupo μ_j: centroide z_i: grupo asignado $|C_j|$: tamaño del grupo

01 Inicializar
Fijar k y elegir k centros iniciales. Usar k-means++ o selección aleatoria.

$$\mu_1, \ldots, \mu_k$$

02 Asignar
Asignar cada observación al centroide más cercano.

$$z_i = \arg\min_{1 \le j \le k} \left\| x_i - \mu_j \right\|^2$$

03 Actualizar
Recalcular cada centroide como la media de las observaciones de su grupo.

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

04 Repetir
Alternar los pasos 2 y 3 hasta estabilizar los centros o agotar las iteraciones.

$$J^{(t+1)} \le J^{(t)}$$

La inercia no aumenta. La convergencia no garantiza el óptimo global.

K-means

Python: ajuste y resultados

Entrada X: matriz numérica con observaciones en filas y variables en columnas.

Implementación

```
from sklearn.cluster import KMeans

# 1. Configurar el modelo
km = KMeans(
    n_clusters=3,
    init="k-means++",
    n_init=10,
    max_iter=300,
    algorithm="lloyd",
    random_state=42)

# 2. Ajustar con los datos
km.fit(X)
```

Qué controla cada decisión

Grupos e inicio
`n_clusters=3` `init="k-means++"`
Fija tres grupos y selecciona los centroides iniciales.

Varios intentos
`n_init=10`
Repite el ajuste diez veces y conserva la menor inercia.

Criterio de parada
`max_iter=300`
Hasta 300 iteraciones por intento.

Resultado reproducible
`random_state=42`
Fija la semilla. Con "lloyd", alterna asignación y actualización.

Después de km.fit(X)

`km.labels_`
Grupo de cada observación.

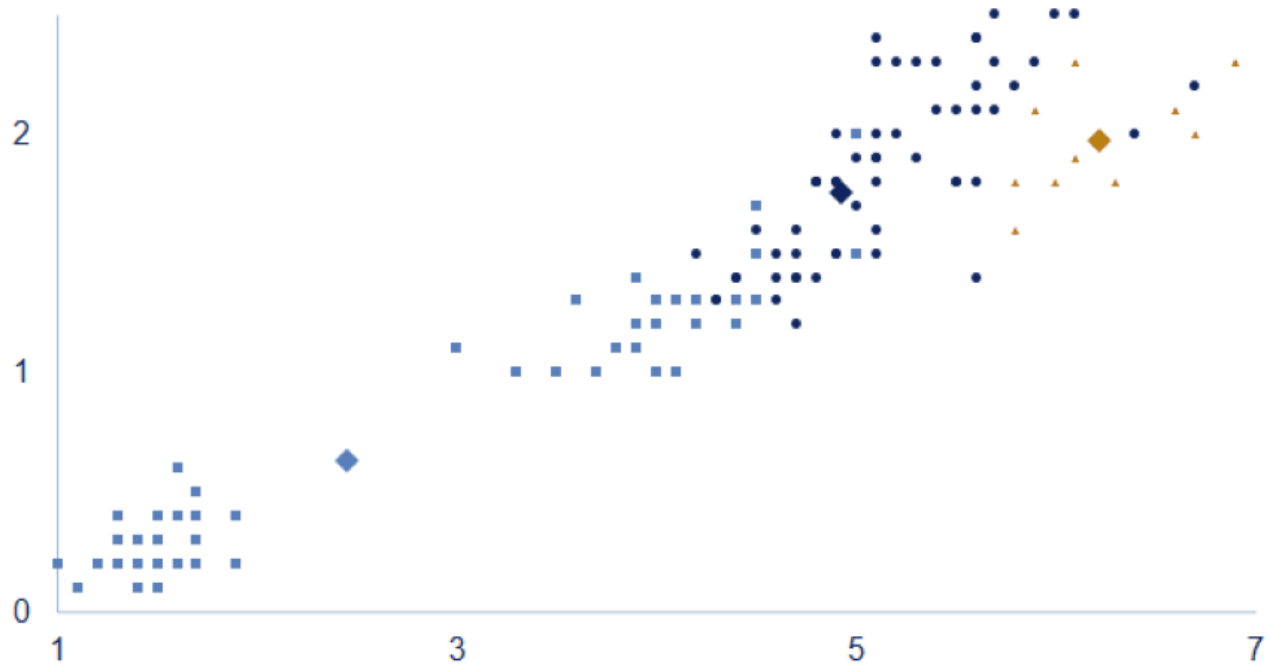
`km.cluster_centers_`
Coordenadas de los centroides.

`km.inertia_`
Suma de distancias² al centroide.

K-means

Algoritmo y evolución de los grupos con Iris

Objetivo: minimizar la inercia interna.
Inicio: fijar k y seleccionar los centros iniciales (aquí, $k = 3$).



Iteración 1

Los colores indican el grupo asignado.

Los rombos grandes son los centroides.

En cada iteración: asignar al centro más próximo y recalcular los centros.
Parada: cambio inferior al umbral o máximo de iteraciones.

K-means

Número de grupos: el codo aplicado a Iris

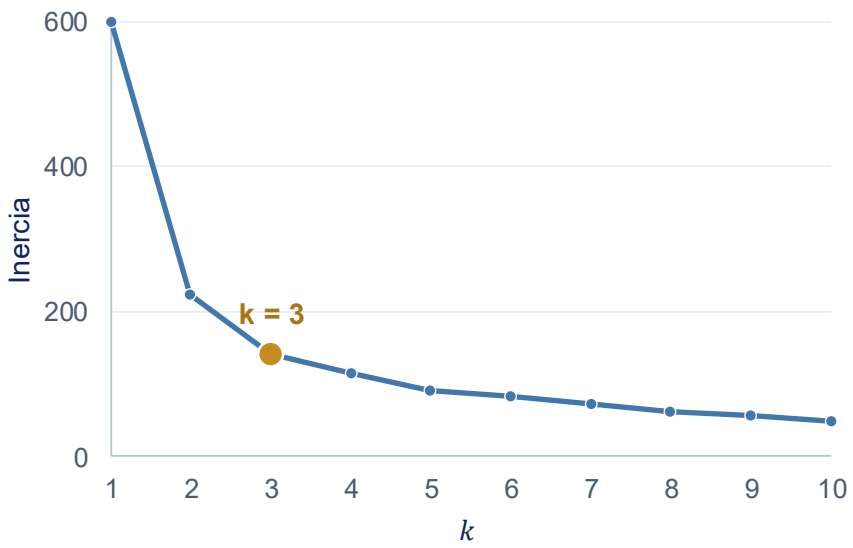
Regla del codo

Ajustar K-means para varios valores de k y representar la inercia (variabilidad intra-clúster) frente a k.

Buscar el cambio de pendiente: a partir de ese punto, añadir grupos reduce mucho menos la inercia.

Ese k es un candidato. La curva no tiene que volverse constante y puede no mostrar un codo claro.

Iris: cuatro variables estandarizadas



k = 3: candidato visual, no óptimo garantizado.

Heurística alternativa: $k \approx \sqrt{n/2}$. Con $n = 800$, $k \approx 20$. Es una orientación basada solo en el tamaño de la muestra, no la regla del codo.

Contrastar la elección con otros indicadores y con la interpretación de los grupos.

Variantes

K-mediods

- La diferencia básica con el K-means es que en vez de escoger el centro de gravedad como centro del clúster, se escoge el **elemento (*mediod*) más representativo**
- No confundir con el algoritmo K-medians que se basa en las medianas univariantes.
- El *mediod* es aquel punto cuya **distancia media al resto de puntos del clúster es mínima**
- Es un algoritmo **más robusto** que el K-means ya que no es tan sensible a la presencia de outliers
- Una variante del algoritmo de **K-mediods** es **CLARA** (*CLustering LARge Applications*) que mejora el rendimiento para grandes volúmenes de datos
- En Python, KMeans de scikit-learn no implementa K-medoids ni CLARA. Requieren otra implementación.

CLUSTERING

Medidas de rendimiento

04

Medidas de rendimiento

Tipos

Internas. Evalúan la calidad de la agrupación sin una referencia externa.
Propiedades deseables:

Compacidad

Baja variabilidad dentro de cada grupo.

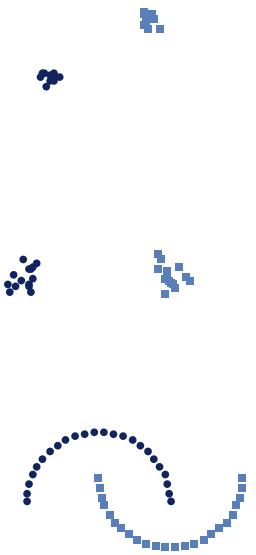
Separación

Alta variabilidad entre grupos.

Conectividad

Hasta qué punto los elementos conexos pertenecen al mismo clúster.

Ejemplos simulados



Externas. Comparan la agrupación con otra agrupación o con un conjunto previamente etiquetado.

Clustering

Medidas internas: una agrupación

Inercia intra

Suma de distancias al cuadrado al centroide. De 0 a ∞. Menor es mejor para un k fijo. El mínimo no aumenta al crecer k.

$$I = \sum_{i=1}^{k} \sum_{j=1}^{n_k} (x_{ij} - c_j)^2$$

Índice de Dunn

Distancia mínima entre grupos / distancia máxima dentro de un grupo. De 0 a ∞. **Mayor es mejor.**

$$\mathrm{D} = \frac{d_{mín}}{d_{máx}}$$

Coeficiente de silueta

De −1 a 1. Mayor es mejor. $a(i)$: distancia media a su grupo. $b(i)$: menor distancia media a otro grupo. Se promedian los $s(i)$.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Python: X y km del ejemplo anterior

Implementación

```
import numpy as np
from sklearn.metrics import (
   pairwise_distances, silhouette_score)
labels = km.labels_
# Inercia del ajuste K-means
inertia = km.inertia_
# Dunn con distancias euclídeas
D = pairwise_distances(X)
same = (labels[:, None] ==
         labels[None, :])
inter = D[~same].min()
intra = D[same].max()
dunn = (inter / intra
        if intra > 0 else np.nan)
# Silueta media
sil = silhouette_score(X, labels)
print(np.round([inertia, dunn, sil], 3))
[139.820  0.026  0.460]
```

Iris estandarizado, $k = 3$. Se requieren entre 2 y $n - 1$ grupos.

Clustering

Medidas externas: comparación de agrupaciones

Comparar las mismas observaciones

Cada pareja se cuenta una vez ($i < j$).

A: juntas en C_1 y en C_2.

B: separadas en C_1 y en C_2.

C: juntas en C_1, separadas en C_2.

D: separadas en C_1, juntas en C_2.

Índice de Rand

Acuerdo entre particiones: cuenta juntas y separadas. De 0 a 1, mayor es mejor.

$$R = \frac{A + B}{A + B + C + D}$$

Jaccard sobre parejas

Cuenta las parejas juntas en ambas entre las juntas en al menos una. De 0 a 1.

$$J = \frac{A}{A + C + D}$$

Python: ejemplo ilustrativo completo

Implementación

```python
import numpy as np
from sklearn.metrics import rand_score
# Etiquetas de las mismas 4 observaciones
C1 = np.array([0, 0, 1, 1])
C2 = np.array([0, 0, 0, 1])
# Sin diagonal ni parejas duplicadas
i, j = np.triu_indices(len(C1), k=1)
s1 = C1[i] == C1[j]
s2 = C2[i] == C2[j]
A = np.sum(s1 & s2)
B = np.sum(~s1 & ~s2)
C = np.sum(s1 & ~s2)
D = np.sum(~s1 & s2)
rand = rand_score(C1, C2)
den = A + C + D
jaccard = A / den if den else 1.0
print(A, B, C, D)
1 2 1 2
print(rand, jaccard)
0.5 0.25
```

Si no hay parejas juntas en ninguna, se toma Jaccard = 1.

CLUSTERING

Sistemas mixtos

05

Sistemas mixtos

K-means + Clusterización jerárquica

- **Opción 1**: Útil con un gran número de observaciones
  - **Etapa 1**. Se realiza un *k-means* con un elevado número de grupos (p.ej, 100)
  - **Etapa 2**. Se realiza una clusterización jerárquica de los grupos usando sus centros de gravedad.
- **Opción 2**: Útil con pocas observaciones. En ciertos casos, mejora el rendimiento
  - **Etapa 1**. Se realiza una clusterización jerárquica
  - **Etapa 2**. Partiendo de una partición inicial de esta clusterización se pueden reasignar los puntos según el algoritmo de k-means

Sistemas mixtos

ACP o ACM con clustering

ACP

Análisis de Componentes Principales

Variables numéricas

ACM

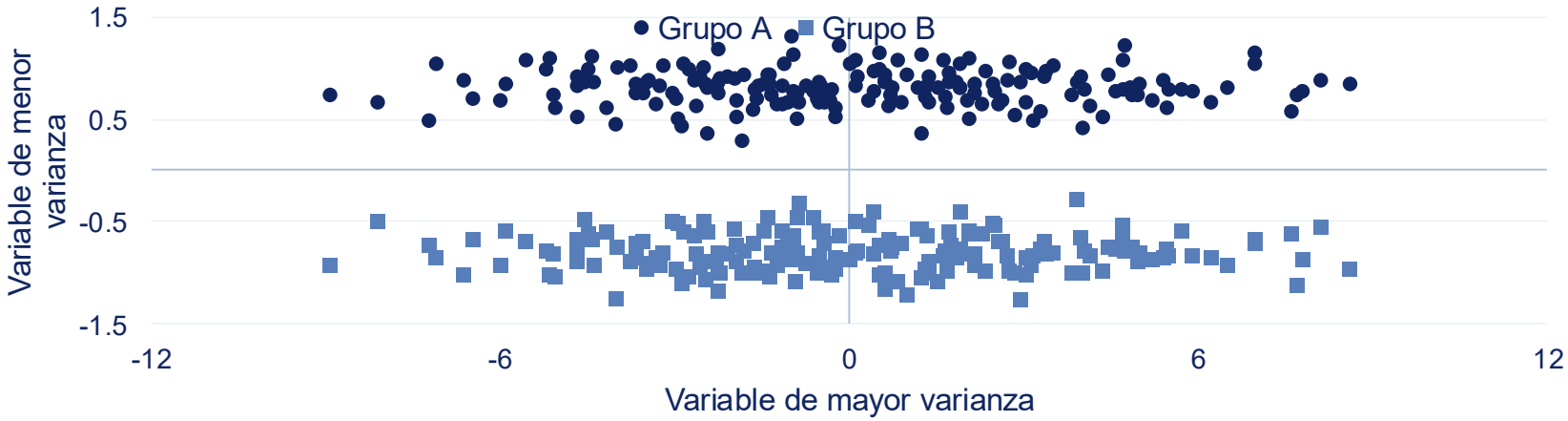
Análisis de Correspondencias Múltiples

Variables categóricas

ACP o ACM puede reducir ruido y representar variables de distintos tipos.

Se retienen las componentes con varianza no nula.

Después se aplica clustering con distancia euclídea.



En algunos casos, el ACP puede ser contraproducente.

ANEXO

DBSCAN

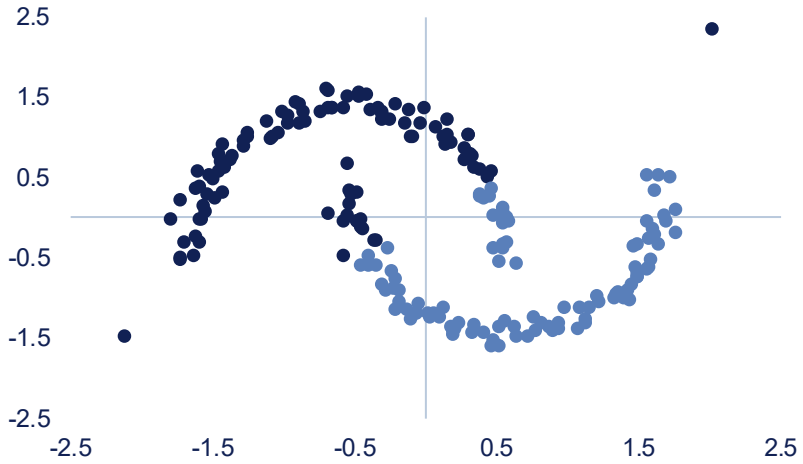
A

Anexo · DBSCAN

Agrupar por densidad, sin fijar el número de grupos

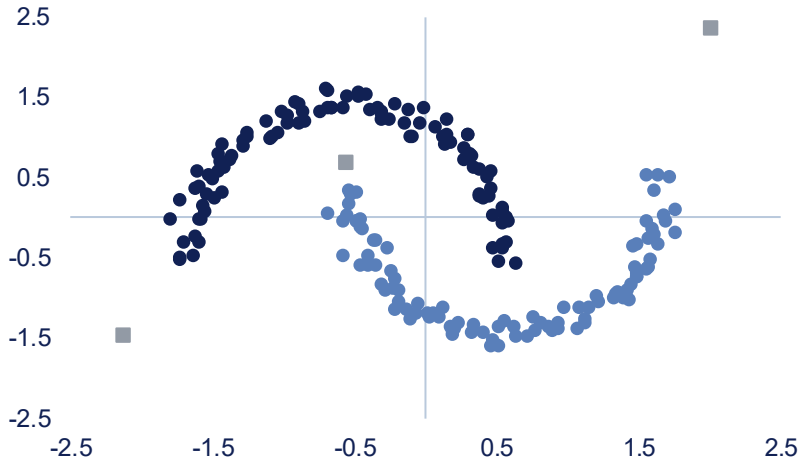
DBSCAN conecta zonas densas y deja como ruido los puntos aislados. Puede recuperar grupos curvos sin imponer un centroide.

K-means: $k = 2$



Asigna todos los puntos. La cercanía a los centroides corta las dos curvas.

DBSCAN: $\varepsilon = 0{,}30$; $\text{min_samples} = 5$



Encuentra 2 grupos y 6 puntos de ruido, marcados con cuadrados grises.

Ejemplo simulado: 205 puntos estandarizados. No se usan las etiquetas originales para ajustar los modelos.

Anexo · DBSCAN

Núcleo, frontera y ruido

ε: radio del vecindario

min_samples: mínimo de puntos en ese radio, incluido el propio punto.

Núcleo

Tiene al menos min_samples puntos en su vecindario. Permite expandir el grupo.

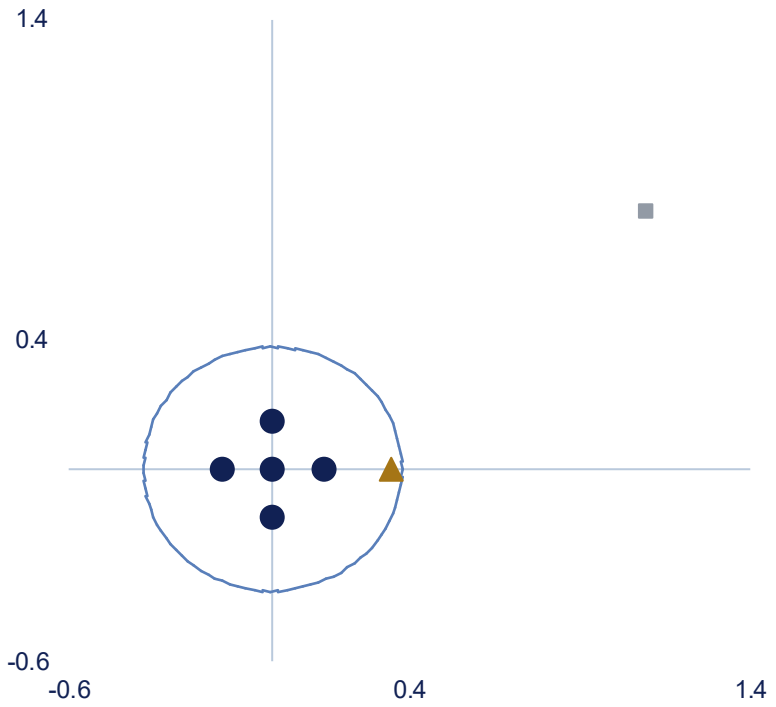
Frontera

No llega al mínimo, pero está a distancia $\leq \varepsilon$ de un núcleo. Pertenece al grupo.

Ruido

No es núcleo ni queda al alcance de ningún núcleo. Recibe la etiqueta −1.

Ejemplo: $\varepsilon = 0{,}38$; $\text{min_samples} = 5$



El círculo muestra el vecindario del punto (0, 0). ε limita la vecindad, no el diámetro del clúster.

Anexo · DBSCAN

Cómo crece un clúster

1 Explorar un punto

Buscar sus vecinos dentro de ε.

2 Comprobar la densidad

Si es núcleo, iniciar un grupo y añadir sus vecinos.

3 Expandir desde los núcleos

Repetir la búsqueda desde cada nuevo núcleo. Los puntos frontera se incorporan, pero no expanden el grupo.

4 Continuar con los demás puntos

Al terminar la expansión, buscar otro núcleo sin asignar. Los puntos que quedan fuera son ruido.

La conexión se propaga

Un grupo puede ser mucho más ancho que ε si una cadena de núcleos lo conecta.

Un punto inicialmente marcado como ruido puede acabar como frontera de un grupo.

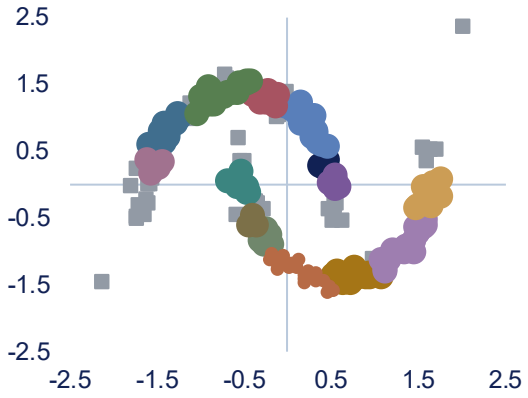
La expansión termina cuando no quedan nuevos núcleos alcanzables. DBSCAN no recalcula centroides.

Anexo · DBSCAN

Elegir ε y min_samples

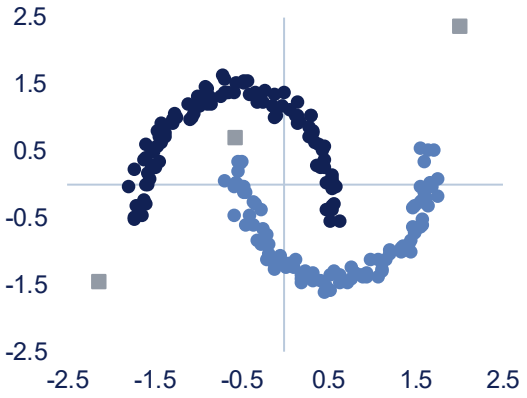
Primero elegir una distancia y una escala coherentes. Después explorar parámetros y comprobar si los grupos y el ruido tienen sentido.

$\varepsilon = 0{,}15$



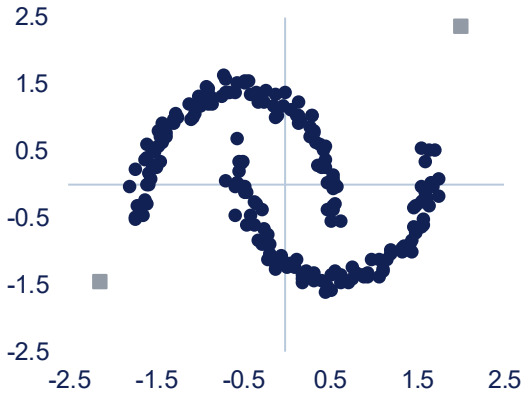
14 grupos; 44 puntos de ruido

$\varepsilon = 0{,}30$



2 grupos; 6 puntos de ruido

$\varepsilon = 0{,}70$



1 grupos; 5 puntos de ruido

ε pequeño puede fragmentar grupos. Grande puede unirlos.

min_samples alto exige regiones más densas para crear núcleos.

Curva k-distancia: $k = \text{min_samples}$ contando el propio punto. Explorar el cambio de pendiente; no garantiza un óptimo.

Anexo · DBSCAN

Python, lectura del resultado y limitaciones

Ejemplo reproducible de las dos curvas

Implementación

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
# 200 puntos y 5 observaciones aisladas
X, _ = make_moons(n_samples=200, noise=.055,
                  random_state=42)
extra = [[-1.5,1.6], [2.3,1.5], [.3,-1.1],
         [1.3,1.6], [-1.4,-.5]]
X = StandardScaler().fit_transform(
    np.vstack([X, extra]))
db = DBSCAN(eps=.3, min_samples=5).fit(X)
labels = db.labels_
# -1 identifica ruido, no un grupo adicional
n_groups = len(set(labels) - {-1})
print(n_groups, np.sum(labels == -1))
2 6
print(len(db.core_sample_indices_))
194
```

Cómo leerlo

2 grupos y 6 puntos de ruido.

194 núcleos y 5 puntos frontera.

Las etiquetas 0 y 1 son arbitrarias.

Límites importantes

Un ε global puede fallar con densidades muy distintas.

La escala y muchas dimensiones dificultan elegir distancias útiles.

En sklearn, la memoria puede llegar a $O(n^2)$.

Ruido significa baja densidad según estos parámetros. No equivale automáticamente a un error o una anomalía real.

MUBD

Màster Universitari en Enginyeria de Dades Massives (Big Data)

Estadística